SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A digital signal processing system produces incorrect results during signed 8-bit integer addition using 2's complement representation when large positive numbers are added. Debug the binary addition process to determine whether overflow has occurred. Recommend an effective overflow detection and handling mechanism to ensure reliable computation.
2. A high-speed embedded controller experiences increased execution time because its arithmetic unit is built with a ripple carry adder. Debug the carry propagation process to identify the cause of the delay. Propose the use of a carry look-ahead adder and explain how it improves the overall arithmetic performance.
3. A processor's signed multiplication module generates incorrect outputs when multiplying negative binary numbers using Booth's algorithm. Debug the algorithm by examining bit-pair evaluation, arithmetic right shifts, and sign extension at each iteration. Identify the fault and recommend modifications to produce accurate multiplication results.
4. A microcontroller performing binary division exhibits poor performance because its restoring division implementation repeatedly restores the partial remainder after unsuccessful subtraction. Debug the sequence of operations to identify redundant steps. Suggest replacing it with the non-restoring division algorithm and explain how the optimization reduces execution time.
5. A numerical computing application produces inaccurate floating-point results when combining values with widely different magnitudes using IEEE 754 single-precision representation. Debug the computation by examining exponent alignment, normalization, and rounding operations. Recommend suitable optimization techniques, such as higher precision or improved numerical methods, to enhance computational accuracy.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

Debugging and Optimization of Computer Arithmetic Operations

1. Debugging Signed 8-bit Integer Addition using 2's Complement

Problem

A digital signal processing system produces incorrect results while adding large positive 8-bit signed integers using 2's complement representation.

Example

Consider adding **+100** and **+60**.

Step 1: Convert to 8-bit Binary

+100 = **01100100**

+60 = **00111100**

Step 2: Binary Addition

```
  01100100
+ 00111100
----------
  10100000
```

Binary result = **10100000**

In 2's complement, this represents **–96**, which is incorrect because

100 + 60 = **160**

The valid range of an 8-bit signed integer is

–128 to +127

Since 160 exceeds +127, **overflow has occurred.**

Identification of Error

- Both operands are positive.
- The result becomes negative.
- This indicates signed overflow.

Debugging Approach and Analysis

1. Verify both operands.
2. Perform binary addition.

3. Observe the sign bits.
4. Compare carry into MSB and carry out of MSB.
5. Detect overflow condition.

Overflow Rule:

- Positive + Positive → Negative = Overflow
- Negative + Negative → Positive = Overflow

Correct Debugged Solution

Implement overflow detection after every signed addition.

Overflow Flag Formula

Overflow = Carry into MSB XOR Carry out of MSB

or

If (Operand1 Sign == Operand2 Sign)

AND

(Result Sign != Operand1 Sign)

→ Overflow

Optimization Technique

- Use dedicated hardware overflow flag.
- Perform saturation arithmetic in DSP applications.

Example

Instead of returning **–96**

Return maximum positive value

127

This prevents unexpected signal distortion.

Testing and Validation

Test Case	**Expected**	**Actual**	**Status**
100 + 20	120	120	Pass
100 + 60	Overflow	Overflow Detected	Pass

Test Case	Expected	Actual	Status
-90 + (-50)	Overflow	Overflow Detected	Pass
40 + 30	70	70	Pass

2. Debugging Ripple Carry Adder Delay

Problem

An embedded controller experiences slow execution because its arithmetic unit uses a Ripple Carry Adder.

Identification of Error

In a Ripple Carry Adder, each full adder must wait for the carry from the previous stage.

Example

FA0 → FA1 → FA2 → FA3 → FA4 → FA5 → FA6 → FA7

The carry propagates through every stage.

This increases propagation delay.

Debugging Approach

Trace carry generation.

Example

A = 11010110

B = 10101101

Carry must pass sequentially through all 8 bits.

Delay

C0

↓

C1

↓

C2

↓

...

↓

C8

This is the main cause of increased execution time.

Correct Debugged Solution

Replace Ripple Carry Adder with Carry Look-Ahead Adder.

Generate and Propagate Signals

Generate (G)

Gi = Ai × Bi

Propagate (P)

Pi = Ai XOR Bi

Carry equations

C1 = G0 + P0C0

C2 = G1 + P1G0 + P1P0C0

...

Carries are generated simultaneously.

Optimization Technique

Advantages of Carry Look-Ahead Adder

- Parallel carry generation
- Faster arithmetic operations
- Lower propagation delay
- Higher processor speed

Complexity

Ripple Carry

O(n)

Carry Look-Ahead

O(log n)

Testing

Compare execution time.

Ripple Carry

8 carry delays

Carry Look-Ahead

1 parallel carry generation

Performance improves significantly.

3. Debugging Booth's Multiplication Algorithm

Problem

Signed multiplication generates incorrect output while multiplying negative numbers.

Example

Multiplicand = -5

Multiplier = 3

Expected Result

-15

Binary Representation

-5

11111011

3

00000011

Identification of Error

Possible faults

- Incorrect bit-pair evaluation
- Missing arithmetic right shift
- Wrong sign extension
- Incorrect accumulator update

Debugging Process

Booth examines

Q0

Q-1

Decision table

Q0	Q-1	Operation
0	0	No operation
0	1	Add Multiplicand
1	0	Subtract Multiplicand
1	1	No operation

After every operation

Perform Arithmetic Right Shift

Maintain sign bit.

Common Fault

Incorrect Logical Right Shift

Example

Before Shift

11110000

Logical Shift

01111000

(Sign Lost)

Correct Arithmetic Shift

11110000

↓

11111000

(Sign Preserved)

Correct Debugged Solution

Ensure

- Correct bit-pair evaluation
- Arithmetic right shift
- Proper sign extension

- Repeat for all n bits

Final Result

11110001

= -15

Correct multiplication obtained.

Optimization

- Use Arithmetic Right Shift only.
- Verify Q0–Q-1 every iteration.
- Extend sign correctly.

Testing

Input	Expected
-5 × 3	-15
-8 × -2	16
7 × -6	-42
9 × 5	45

All outputs should match expected values.

4. Debugging Restoring Division

Problem

Restoring Division performs unnecessary restore operations, increasing execution time.

Example

Dividend = 13

Divisor = 3

Identification of Error

Restoring Division steps

1. Shift
2. Subtract
3. Negative?

4. Restore
5. Continue

Restoration happens repeatedly.

This wastes processor cycles.

Debugging Analysis

Flow

Shift

↓

Subtract

↓

Negative?

↓

Restore

↓

Repeat

Frequent restoring causes delay.

Correct Debugged Solution

Replace Restoring Division with Non-Restoring Division.

Algorithm

If remainder is positive

Subtract divisor

If remainder is negative

Add divisor next cycle

No immediate restoration required.

Optimization

Advantages

- Fewer arithmetic operations
- Reduced execution time

- Better processor efficiency
- Faster division

Testing

Example

13 ÷ 3

Expected

Quotient = 4

Remainder = 1

Both algorithms produce the same answer.

Non-Restoring Division uses fewer operations.

5. Debugging IEEE 754 Floating Point Computation

Problem

Combining numbers with very different magnitudes causes inaccurate floating-point results.

Example

100000000 + 1

IEEE 754 Single Precision

Identification of Error

Large number

1.0 × 2^26

Small number

1.0 × 2^0

During exponent alignment

The smaller number shifts right many times.

Its least significant bits disappear.

This causes precision loss.

Debugging Approach

Check

1. Exponent alignment
2. Mantissa shifting
3. Normalization
4. Rounding

Loss occurs during mantissa alignment.

Correct Debugged Solution

Use

- Double precision (64-bit)
- Kahan Summation Algorithm
- Pairwise summation
- Appropriate rounding modes

These reduce floating-point errors.

Optimization Techniques

- IEEE 754 Double Precision
- Higher precision arithmetic
- Numerical stability algorithms
- Fused Multiply-Add (FMA)
- Compensated summation

These improve computational accuracy.

Testing

Expression	Single Precision	Improved Method
100000000 + 1	100000000	100000001
0.1 + 0.2	0.30000001	0.30000000
1e20 + 1 − 1e20	0	1

Improved methods provide more accurate results.

Conclusion

The debugging process identified arithmetic errors related to signed overflow, carry propagation delay, Booth's multiplication, restoring division, and IEEE 754 floating-point

computation. Appropriate solutions such as overflow detection, Carry Look-Ahead Adders, correct arithmetic right shifts with sign extension, Non-Restoring Division, and higher-precision numerical methods improve both correctness and performance. These optimizations ensure reliable arithmetic operations in modern processors and embedded systems.